

Construcción y Despliegue de Rutas Dinámicas en Express

El equivalente práctico a un servlet, en
TypeScript

Ing. Alberto Manases Turcios Ortiz
Gestión de Servidores Web – Sección A
Miércoles 12 de agosto de 2026



Encuadre de la clase

Unidad 1: Creación de contenedores web para la ejecución de servlets

Horario: 1:00 pm – 2:40 pm

Duración: 1 hora 40 minutos

Objetivos de la clase

1. Explicar la equivalencia entre un "servlet" (terminología oficial) y un *route handler* de Express en TypeScript.
2. Escribir múltiples rutas que reciben parámetros y devuelven respuestas calculadas.
3. Desplegar los cambios en el servicio que dejamos corriendo en la Clase 7.



Conexión con la Clase 7

Recordatorio: en la Clase 7 instalamos Node.js, TypeScript y Express, y dejamos un servicio corriendo con una sola ruta (/).

Hoy: le damos uso real al contenedor – esto es, en esencia, lo mismo que en la Clase 2 hicimos con `http.createServer`, pero con una herramienta profesional (Express) que nos ahorra trabajo repetitivo.

Si su servicio de la Clase 7 no quedó corriendo, este es el momento de retomarlo – todo lo de hoy se construye encima de `servidor.ts`.



Servlet = route handler: el mismo concepto

Concepto (terminología oficial)	Cómo se ve en Express (TypeScript)
Servlet / route handler	Función registrada con <code>app.get</code> , <code>app.post</code> , etc.
Ruta registrada en el contenedor	<code>app.get("/ruta", ...)</code>
Función que atiende la petición	<code>(req: Request, res: Response) => {...}</code>
El contenedor administra el ciclo de vida	Express + Node.js administran el ciclo de vida



¿Por qué Express y no solo `http.createServer`?

Recuerden la Clase 2: tuvimos que escribir manualmente el `if (url.pathname === ...)` para **cada** ruta.

Lo que Express automatiza

- El `if` repetitivo por cada ruta – ahora es una línea: `app.get('/ruta', ...)`.
- Manejo de parámetros de la URL (`req.query`).
- Manejo de errores y de múltiples métodos HTTP (GET, POST, etc.) de forma ordenada.



Ruta 1: /saludo – saludo personalizado

```
1 // Ruta 1: saludo personalizado (el route handler mas simple posible)
2 app.get("/saludo", (req: Request, res: Response) => {
3   const nombre = (req.query.nombre as string) ?? "desconocido";
4   res.send(`<h1>Hola, ${nombre}!</h1>`);
5 });
```

req.query.nombre lee el valor después del ? en la URL – es exactamente lo que hacíamos con `url.searchParams.get(...)` en la Clase 2, pero Express ya lo deja listo en `req.query`.



Ruta 2: /info – información calculada al momento

```
1 // Ruta 2: informacion del servidor en el momento de la peticion
2 app.get("/info", (req: Request, res: Response) => {
3   res.json({
4     fechaServidor: new Date().toISOString(),
5     userAgent: req.headers["user-agent"],
6   });
7 });
```

res.json(...) hace dos cosas que antes hacíamos a mano: convierte el objeto a texto JSON y pone el Content-Type correcto. *new Date()* – igual que en la Clase 3 – prueba que esto se calcula en cada petición, no es un archivo estático.



Ruta 3: /cotizacion – cálculo con validación

```
1 // Ruta 3: calculo dinamico -- total con IVA (13%, El Salvador)
2 app.get("/cotizacion", (req: Request, res: Response) => {
3   const monto = parseFloat((req.query.monto as string) ?? "0");
4   if (isNaN(monto) || monto < 0) {
5     res.status(400).json({ error: "monto invalido" });
6     return;
7   }
8   const totalConIva = monto * 1.13;
9   res.json({ montoOriginal: monto, iva: monto * 0.13, total: totalConIva });
10 });
```

*El if que valida **antes** de calcular es nuevo respecto a las clases anteriores – y es la parte más importante de esta ruta.*



¿Por qué validar antes de calcular?

Un contenedor que confía ciegamente en el usuario es inseguro

Si alguien pide `/cotizacion?monto=hola` o `/cotizacion?monto=-5`, sin el `if` de validación el servidor intentaría calcular IVA sobre algo que no es un número válido – resultado: respuestas absurdas o el proceso completo **se cae**.

Código de estado HTTP 400: significa "Bad Request" – "tu petición está mal formada, no es que yo haya fallado". Es la forma correcta de rechazar datos inválidos.

Este es apenas un primer asomo a seguridad – lo profundizamos en la Clase 9.



Recompilar y reiniciar el servicio

Mismos comandos de compilación de siempre, y luego reiniciar el servicio que dejamos en la Clase 7 según tu sistema:

```
1 tsc servidor.ts
```

Linux

```
1 sudo systemctl \
2   restart miapp
3 sudo systemctl \
4   status miapp
```

macOS

```
1 sudo launchctl unload \
2   /Library/LaunchDaemons/com.miapp.
3   plist
4 sudo launchctl load -w \
5   /Library/LaunchDaemons/com.miapp.
6   plist
```

Windows

```
1 nssm restart miapp
2 Get-Service miapp
```

systemd tiene restart directo; launchd necesita unload + load para releer el programa actualizado – son formas distintas de pedirle lo mismo al sistema: "olvida la versión vieja, corre la nueva".



Probar cada ruta

URL	Resultado esperado
/saludo?nombre=Estudiante	Hola, Estudiante!
/info	JSON con fecha y user-agent
/cotizacion?monto=100	JSON con monto, IVA y total
/cotizacion?monto=-5	Error controlado (HTTP 400)

*Probar también /cotizacion **sin** el parámetro monto – van a ver el mismo error controlado, porque ?? "0" convierte la ausencia del parámetro en 0, y la validación lo atrapa igual.*



Ejercicio individual

1. Completar el despliegue de las 3 rutas y probarlas todas.
2. Agregar una cuarta ruta `/factorial` que reciba un parámetro `n` y devuelva el factorial de ese número (repaso de lógica de programación).
3. **Reto adicional:** hacer que `/saludo` también acepte el método `POST`, recibiendo el nombre en el cuerpo (*body*) de la petición en lugar de la URL.

Pista para el reto: van a necesitar `app.use(express.json())` antes de definir la ruta, para que Express sepa leer el body en formato JSON.



Pista para el ejercicio

Pista

`/factorial` sigue **el mismo molde** que `/cotizacion`: leer parámetro → validar (¿es un número? ¿es negativo?) → calcular → `res.json(...)`. El factorial se calcula con un ciclo simple multiplicando de 1 hasta `n`.

No olviden recompilar (`tsc servidor.ts`) y reiniciar el servicio después de cada cambio – si prueban y ven el código viejo, esa es la causa más común.



Verificación de aprendizaje

1. ¿Cuál es el equivalente, en nuestro curso, a un "servlet" (terminología oficial de contenedores web)?





Verificación de aprendizaje

1. ¿Cuál es el equivalente, en nuestro curso, a un "servlet" (terminología oficial de contenedores web)?

Una función route handler registrada con `app.get/app.post` en Express.





Verificación de aprendizaje

1. ¿Cuál es el equivalente, en nuestro curso, a un "servlet" (terminología oficial de contenedores web)?

Una función route handler registrada con `app.get/app.post` en Express.

2. ¿Qué hace `req.query.nombre`?





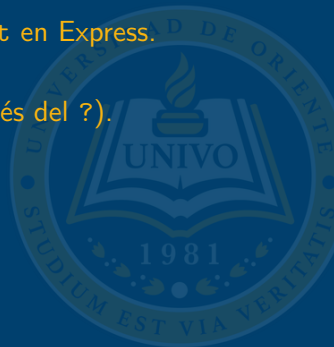
Verificación de aprendizaje

1. ¿Cuál es el equivalente, en nuestro curso, a un "servlet" (terminología oficial de contenedores web)?

Una función route handler registrada con `app.get/app.post` en Express.

2. ¿Qué hace `req.query.nombre`?

Lee el valor del parámetro `nombre` enviado en la URL (después del ?).





Verificación de aprendizaje

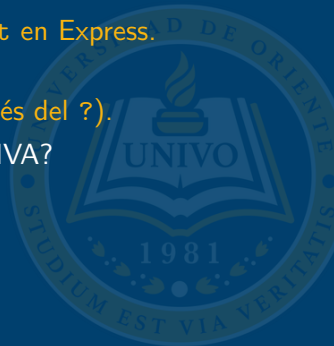
1. ¿Cuál es el equivalente, en nuestro curso, a un "servlet" (terminología oficial de contenedores web)?

Una función route handler registrada con `app.get/app.post` en Express.

2. ¿Qué hace `req.query.nombre`?

Lee el valor del parámetro `nombre` enviado en la URL (después del ?).

3. ¿Por qué validamos el parámetro `monto` antes de calcular el IVA?





Verificación de aprendizaje

1. ¿Cuál es el equivalente, en nuestro curso, a un "servlet" (terminología oficial de contenedores web)?

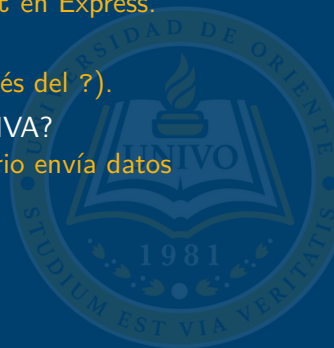
Una función route handler registrada con `app.get/app.post` en Express.

2. ¿Qué hace `req.query.nombre`?

Lee el valor del parámetro `nombre` enviado en la URL (después del ?).

3. ¿Por qué validamos el parámetro `monto` antes de calcular el IVA?

Para evitar errores o comportamientos inesperados si el usuario envía datos inválidos.





Resumen de la clase

Paso	Acción/código
Definir ruta	<code>app.get(/ruta", (req, res) => {...})</code>
Leer parámetro	<code>req.query.nombre</code>
Responder JSON	<code>res.json({...})</code>
Recompilar y aplicar	<code>tsc servidor.ts + reiniciar servicio</code>



Tarea para casa

1. Crear una quinta ruta `/temperatura` que convierta grados Celsius a Fahrenheit, recibiendo `celsius` como parámetro.
2. Documentar el proceso completo (código, compilación, despliegue, pruebas) con capturas.

Para profundizar (documentación oficial)

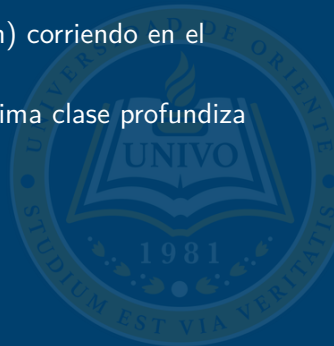
- expressjs.com – guía de routing
- developer.mozilla.org – códigos de estado HTTP



Preparación para la Clase 9

Tema: Buenas prácticas y seguridad en contenedores web.

- Dejar las 3 rutas de hoy (más `/factorial`, si lo completaron) corriendo en el servicio.
- Repasar por qué validamos monto antes de calcular – la próxima clase profundiza exactamente esa idea.





Comisión de Acreditación
de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

MUCHAS GRACIAS

Te espero la próxima clase

